

Apprentissage par renforcement : prédiction et contrôle Model Free

Marin Ferecatu

Notes de cours

UE RCP211, Séance RL No. 2

CNAM Paris

Document rédigé en L^AT_EX

29 mars 2026

Table des matières

Avant-propos	3
Notations globales	4
1 Introduction	5
1.1 Pourquoi passer au model-free ?	5
1.2 Le cadre général model-free en grandes lignes	5
2 Prédiction Model Free	7
2.1 Échantillonnage de Monte Carlo	7
2.1.1 But du Monte Carlo sampling	7
2.1.2 First-Visit Monte Carlo Policy Evaluation	7
2.1.3 Every-Visit Monte Carlo Policy Evaluation	8
2.1.4 Moyenne incrémentale et Monte Carlo incrémental	8
2.1.5 Forces et limites des méthodes Monte Carlo	9
2.2 Différence Temporelle TD(0)	9
2.2.1 De Monte Carlo à TD(0) : intuition	9
2.2.2 TD(0)	10
2.2.3 Offline vs online	11
2.2.4 Généralisation à n -step TD(n)	12
2.2.5 Exemples classiques pour TD	12
2.2.6 Monte Carlo versus TD(0)	18
2.3 TD(λ)	18
2.3.1 Transition depuis le n -step TD	18
2.3.2 Idée générale	18
2.3.3 Vue forward : combinaison de tous les retours à plusieurs pas	19
2.3.4 Vue <i>backward</i> et traces d'éligibilité	20
3 Contrôle Model Free	22
3.1 Introduction : contrôle versus prédiction	22
3.1.1 Generalized Policy Iteration	22

3.1.2	Politique ε -gloutonne et GLIE	22
3.1.3	GLIE Monte Carlo	23
3.2	<i>On-policy</i> et SARSA	23
3.2.1	Le problème <i>on-policy</i>	23
3.2.2	Algorithme SARSA	24
3.2.3	Pourquoi SARSA est-il <i>on-policy</i> ?	24
3.2.4	Garanties mathématiques : idée générale	24
3.2.5	Extensions de SARSA	25
3.2.6	Similitudes avec TD(0)	25
3.3	<i>Off-policy</i> et Q-Learning	25
3.3.1	<i>Off-policy</i> learning	25
3.3.2	Importance sampling	25
3.3.3	<i>Off-policy</i> Monte Carlo	26
3.3.4	<i>Off-policy</i> TD	27
3.3.5	Q-Learning	27
3.3.6	Garanties mathématiques : idée générale	29
3.3.7	SARSA versus Q-learning : l'exemple de la falaise	29
3.3.8	Autres exemples classiques pour le TD de contrôle	30
3.3.9	Vers l'approximation de fonction et le Deep RL	31
	Conclusion	32
	Références	32

Avant-propos

Ces notes poursuivent directement le document *Apprentissage par renforcement : cadre général et programmation dynamique* [1]. On y suppose connus le cadre markovien, les notions de retour, de fonction de valeur, de politique, ainsi que les équations de Bellman et les méthodes de programmation dynamique du document précédent. Lorsque cela sera utile, on renverra explicitement à ce premier texte, par exemple à la Section 2.10 sur les équations de Bellman ou à la Section 3.6 sur la transition vers le model-free RL [1].

L'idée générale est la suivante. Dans le premier document, le modèle de l'environnement était supposé connu : on connaissait les probabilités de transition $P(s' | s, a)$ et les récompenses $r(s, a, s')$, ce qui permettait de calculer des espérances exactes. Ici, on supprime précisément cette hypothèse. Les mêmes quantités mathématiques restent pertinentes, mais on ne peut plus les calculer par sommes exactes sur tous les états suivants ; il faut les approximer à partir de trajectoires observées.

Le changement est à la fois conceptuel et algorithmique. Conceptuellement, les objets à apprendre restent les fonctions v_π , q_π , parfois v_* ou q_* . Du point de vue algorithmique, on passe d'équations résolues exactement à des mises à jour fondées sur des échantillons, donc sur de l'aléa. C'est ce déplacement du calcul exact vers l'estimation à partir de l'expérience qui constitue le coeur du *model-free reinforcement learning*.

Notations globales

Les notations du premier polycopié sont conservées. On en ajoute quelques-unes, spécifiques aux méthodes model-free.

G_t	retour (<i>return</i>) à partir de l'instant t ;
$V_t(s)$	estimation courante de $v_\pi(s)$ ou de $v_*(s)$;
$Q_t(s, a)$	estimation courante de $q_\pi(s, a)$ ou de $q_*(s, a)$;
α_t ou α	pas d'apprentissage (<i>step size, learning rate</i>) ;
$N(s)$	nombre de visites de l'état s ;
$N(s, a)$	nombre de visites du couple (s, a) ;
δ_t	erreur TD (<i>TD error</i>) à l'instant t ;
ε	paramètre d'exploration d'une politique ε -gloutonne ;
$\lambda \in [0, 1]$	paramètre de traces de TD(λ) ;
b	politique de comportement (<i>behavior policy</i>) ;
μ	politique cible (<i>target policy</i>) dans le cadre off-policy.

Comme dans le document précédent, le temps est discret. Dans les chapitres de contrôle, on travaillera souvent avec des estimations de valeurs d'état-action, car elles permettent de comparer directement les actions sans connaître le modèle.

Chapitre 1

Introduction

1.1 Pourquoi passer au model-free ?

Dans le document précédent, les méthodes de programmation dynamique supposaient connu le quadruplet $(\mathcal{S}, \mathcal{A}, P, r)$ du MDP. Cette hypothèse est très forte. Dans beaucoup de situations réalistes, on ne connaît ni les probabilités de transition ni même les récompenses moyennes associées à chaque transition. On sait seulement faire interagir l'agent avec l'environnement et observer ce qui se passe.

Le terme **model-free** (sans modèle) signifie précisément ceci : on ne dispose pas d'un modèle explicite de l'environnement sous la forme de $P(s' | s, a)$ et $r(s, a, s')$, et l'on ne cherche pas d'abord à les estimer pour ensuite résoudre les équations de Bellman. On apprend plus directement les quantités utiles à la décision, en particulier les fonctions de valeur et les politiques.

Il faut distinguer soigneusement deux idées.

Dans une approche **model-based** (avec modèle), on connaît ou l'on apprend un modèle de la dynamique, puis on raisonne à l'intérieur de ce modèle. C'est le cadre de la programmation dynamique étudié dans le premier document.

Dans une approche **model-free**, on part de trajectoires observées,

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots,$$

et l'on essaie d'en extraire directement une bonne estimation des valeurs ou une bonne politique.

La différence centrale est donc la suivante.

- En model-based, on calcule des espérances exactes à partir du modèle.
- En model-free, on remplace ces espérances par des moyennes empiriques ou par des corrections fondées sur des transitions effectivement observées.

Cette idée prolonge directement la section 3.6 du document précédent, où l'on expliquait que les équations de Bellman resteraient valables comme fondation conceptuelle, mais qu'il faudrait désormais les approcher à partir d'échantillons [1, section 3.6, p. 29].

1.2 Le cadre général model-free en grandes lignes

Le cadre model-free peut se décrire en répondant à quelques questions simples.

Quelles sont les données d'apprentissage ? Ce ne sont pas des couples entrée-sortie comme en apprentissage supervisé. Ce sont des trajectoires, partielles ou complètes, produites par interaction avec l'environnement : états visités, actions choisies, récompenses observées et états suivants.

Comment obtient-on ces données ? Elles sont obtenues en exécutant une politique. Cette politique peut être fixée si l'on fait seulement de la prédiction, ou modifiée progressivement si l'on cherche le contrôle. Les données dépendent donc des choix de l'agent ; elles ne sont ni indépendantes ni données à l'avance.

Quel est le but ? Dans un problème de *prédiction* (prediction), le but est d'évaluer une politique donnée, c'est-à-dire d'estimer v_π ou q_π . Dans un problème de *contrôle* (control), le but est de construire une politique optimale, ou aussi proche que possible de l'optimalité. En pratique, on cherche souvent à approximer la fonction de valeur optimale d'action q_* , car une politique gloutonne (greedy) par rapport à q_* est optimale.

Comment se déroule l'apprentissage ? On répète des épisodes ou des transitions, on met à jour les estimations à partir des observations, puis éventuellement on améliore la politique. Le schéma général est donc : collecter de l'expérience, mettre à jour des valeurs, améliorer le comportement, recommencer.

Qu'obtient-on à la fin ? Suivant les méthodes, on obtient une estimation de valeur V ou Q , une politique explicite, ou les deux. Dans les méthodes de contrôle les plus classiques, une bonne estimation de Q suffit en pratique, car il devient alors naturel de choisir dans chaque état l'action de plus grande valeur estimée.

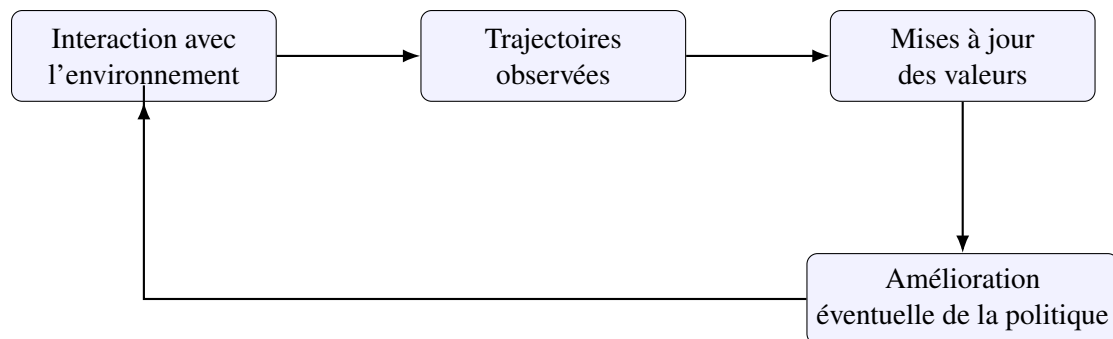


FIGURE 1.1 – Boucle générale d'apprentissage en RL model-free.

Deux observations sont essentielles. D'abord, le signal disponible est souvent bruité : deux visites du même état peuvent produire des retours différents. Ensuite, dans un problème de contrôle, la politique qui collecte les données change souvent en cours d'apprentissage. Le problème est donc intrinsèquement adaptatif.

Chapitre 2

Prédiction Model Free

2.1 Échantillonnage de Monte Carlo

2.1.1 But du Monte Carlo sampling

Les méthodes de **Monte Carlo** (Monte Carlo methods) cherchent à estimer une espérance en la remplaçant par une moyenne d'observations. Dans le contexte du RL, l'idée est naturelle : puisque

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s],$$

on peut essayer d'estimer $v_{\pi}(s)$ en observant plusieurs fois un retour G_t à partir de l'état s et en en faisant la moyenne.

Le point important est que l'on ne remplace pas ici une somme sur les états suivants, comme dans Bellman, par une transition unique ; on attend au contraire la fin de l'épisode pour utiliser un *retour complet*. Les méthodes Monte Carlo sont donc fondées sur l'idée suivante :

si l'on suit toujours la politique π , alors les retours observés après les visites d'un même état finissent par fournir une bonne estimation de sa valeur moyenne.

Cette approche demande typiquement des tâches *épisodiques* (episodic tasks), c'est-à-dire des problèmes où l'on peut parler naturellement d'un début et d'une fin d'épisode. En effet, pour calculer le retour complet G_t , il faut savoir où s'arrête la trajectoire considérée.

2.1.2 First-Visit Monte Carlo Policy Evaluation

Définition 2.1 (First-visit Monte Carlo policy evaluation). L'**évaluation de politique Monte Carlo first-visit** consiste, pour chaque état s , à moyenner les retours observés après la *première visite* de s dans chaque épisode généré par la politique π .

Soit un épisode

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T,$$

où S_T est terminal. Pour chaque instant $t < T$, on définit le retour

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T.$$

Si $S_t = s$ et si c'est la première fois que l'état s apparaît dans l'épisode, on ajoute G_t à la liste des retours associés à s . L'estimation de $v_{\pi}(s)$ est ensuite la moyenne de tous les retours ainsi collectés.

Pourquoi ne retenir que la première visite ? L'idée est d'éviter de compter plusieurs fois, dans un même épisode, des observations fortement dépendantes. Ce n'est pas indispensable pour la convergence, mais c'est historiquement la variante la plus simple à présenter.

Sous des hypothèses standard (politique fixée, épisodes indépendants au sens des redémarrages, retour borné), la loi des grands nombres suggère que la moyenne empirique des retours first-visit converge vers $v_\pi(s)$.

Remarque 2.2. Le Monte Carlo first-visit ne nécessite aucun modèle. On n'a jamais besoin de connaître $P(s' | s, a)$ ni $r(s, a, s')$. Il suffit de générer des épisodes en suivant π .

2.1.3 Every-Visit Monte Carlo Policy Evaluation

Définition 2.3 (Every-visit Monte Carlo policy evaluation). L'évaluation de politique Monte Carlo **every-visit** consiste à moyenner les retours observés après *toutes* les visites d'un état s dans les épisodes générés par la politique π .

Cette variante paraît encore plus naturelle : chaque fois que l'on voit l'état s , on peut calculer le retour complet qui suit cette visite et l'utiliser comme observation de la quantité aléatoire $G_t | S_t = s$.

La différence avec first-visit n'est donc pas conceptuelle, mais pratique.

- **First-visit** : un état contribue au plus une fois par épisode.
- **Every-visit** : un état peut contribuer plusieurs fois dans un même épisode.

Les deux variantes convergent sous des hypothèses raisonnables vers la même valeur $v_\pi(s)$. En pratique, every-visit exploite davantage les données disponibles, tandis que first-visit est parfois plus simple à analyser et à expliquer.

2.1.4 Moyenne incrémentale et Monte Carlo incrémental

Si l'on stocke tous les retours observés pour chaque état, la méthode est simple mais peu élégante. Heureusement, une moyenne peut se mettre à jour de manière incrémentale.

Supposons avoir observé successivement des quantités $x_1, x_2, \dots, x_n$ de moyenne

$$\bar{x}_n = \frac{1}{n} \sum_{k=1}^n x_k.$$

Alors

$$\bar{x}_{n+1} = \bar{x}_n + \frac{1}{n+1} (x_{n+1} - \bar{x}_n).$$

Cette relation est fondamentale. Elle montre qu'une nouvelle observation corrige l'estimation précédente dans la direction de l'erreur commise. Appliquée au Monte Carlo, elle donne pour un état s :

$$V(s) \leftarrow V(s) + \frac{1}{N(s)} (G_t - V(s)),$$

où $N(s)$ est le nombre de retours déjà utilisés pour cet état.

On peut aussi remplacer $1/N(s)$ par une constante $\alpha \in (0, 1]$:

$$V(s) \leftarrow V(s) + \alpha (G_t - V(s)).$$

La première version donne une vraie moyenne empirique. La seconde oublie progressivement le passé et devient utile lorsque l'environnement n'est pas parfaitement stationnaire ou lorsque l'on veut réagir plus vite aux nouvelles données.

Remarque 2.4. Le schéma

$$\text{nouvelle estimation} = \text{ancienne estimation} + \text{pas} \times \text{erreur}$$

reviendra sans cesse dans tout le chapitre. Monte Carlo, TD, SARSA et Q-learning partagent tous cette structure générale.

Algorithme : First-Visit Monte Carlo

Entrée : une politique fixée π et une estimation initiale V de la fonction de valeur (par exemple $V(s) = 0$ pour tout état s).

1. Générer un épisode complet en suivant π .
2. Pour chaque état s apparaissant dans l'épisode, repérer sa première visite.
3. Calculer le retour G_t à partir de cette visite.
4. Mettre à jour $V(s)$ par moyenne empirique ou moyenne incrémentale.
5. Recommencer avec un nouvel épisode.

L'algorithme *Every-Visit Monte Carlo* est identique, sauf qu'au lieu de ne retenir, pour chaque état s , que sa première visite dans l'épisode, on utilise chacune de ses visites. Autrement dit, on calcule un retour G_t pour tout instant t tel que l'état S_t apparaît dans l'épisode, et chacun de ces retours contribue à la mise à jour de l'estimation de $V(S_t)$.

On peut remarquer que, dans la variante *Every-Visit*, un même état peut apparaître plusieurs fois dans un épisode, et que les retours associés sont alors calculés sur des suffixes de trajectoire de longueurs différentes. Cela ne pose pas de difficulté conceptuelle : à chaque instant t tel que $S_t = s$, le retour G_t est précisément un échantillon du retour futur à partir de l'état s , c'est-à-dire de la quantité dont l'espérance vaut $v_\pi(s)$.

Remarque. Lorsque l'on utilise la variante *Every-Visit*, les retours associés aux différentes visites d'un même état dans un épisode sont généralement fortement corrélés, car ils proviennent de suffixes emboîtés de la même trajectoire. De plus, une même récompense peut intervenir dans plusieurs retours, tandis que son influence dépend aussi du facteur d'actualisation γ : plus une récompense est lointaine, moins son poids est important lorsque $\gamma < 1$.

2.1.5 Forces et limites des méthodes Monte Carlo

Les méthodes Monte Carlo ont plusieurs qualités pédagogiques et conceptuelles. Elles sont simples, directement reliées à la définition de v_π comme espérance du retour, et elles ne nécessitent aucun bootstrap, c'est-à-dire aucune utilisation d'une estimation pour corriger une autre estimation.

Elles ont aussi des limites importantes.

- Il faut attendre la fin de l'épisode avant de mettre à jour.
- La variance des retours peut être grande, surtout lorsque les épisodes sont longs.
- Les tâches continues, sans état terminal naturel, y sont moins commodes.

Ces limites motivent le passage aux méthodes de différence temporelle.

2.2 Différence Temporelle TD(0)

2.2.1 De Monte Carlo à TD(0) : intuition

Dans le 1er document de notes de cours, l'équation de Bellman pour une politique fixée était [1, section 2.10]

$$v_\pi(s) = \mathbb{E}_\pi [R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s].$$

Monte Carlo utilise une réalisation complète du retour G_t pour approcher $v_\pi(s)$. L'idée de **Temporal-Difference learning** (apprentissage par différence temporelle) est plus locale : au lieu d'attendre la fin de l'épisode, on remplace le futur complet par une estimation déjà disponible de la valeur de l'état suivant.

Autrement dit, au lieu de viser directement la cible aléatoire

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots,$$

on utilise la cible à un pas

$$R_{t+1} + \gamma V(S_{t+1}).$$

Cette quantité est plus grossière que le retour complet, mais elle est disponible immédiatement après une seule transition.

On parle de **bootstrap** parce que l'on corrige une estimation $V(S_t)$ à l'aide d'une autre estimation, ici $V(S_{t+1})$. C'est précisément ce qui distingue TD des méthodes Monte Carlo.

Concrètement, on peut voir la méthode TD comme un procédé de mise à jour progressive d'une table de valeurs. À chaque état s accessible, on associe une estimation courante $V(s)$ de sa valeur. Au début de l'apprentissage, ces quantités sont simplement initialisées, par exemple à 0, ou à n'importe quelles valeurs choisies par l'utilisateur. Elles ne coïncident en général pas avec les vraies valeurs $v_\pi(s)$.

L'apprentissage consiste alors à modifier progressivement ces estimations au fur et à mesure des transitions observées. Chaque nouvelle donnée réelle de la forme

$$S_t = s, \quad R_{t+1} = r, \quad S_{t+1} = s'$$

déclenche une mise à jour de l'estimation $V(s)$. On utilise pour cela l'information immédiatement disponible, à savoir la récompense observée r et l'estimation courante $V(s')$ de l'état suivant. Ainsi, les premières mises à jour peuvent être fortement influencées par l'initialisation, puisque toutes les valeurs stockées sont encore approximatives. Mais, à mesure que l'agent accumule de l'expérience et que les corrections se répètent, les estimations deviennent plus cohérentes entre elles et se rapprochent progressivement des vraies valeurs.

Autrement dit, dans TD, on n'attend pas de calculer une valeur exacte avant de l'utiliser : on travaille en permanence avec les meilleures estimations disponibles à l'instant considéré, et on les améliore petit à petit.

2.2.2 TD(0)

Avant d'énoncer la règle de mise à jour, introduisons la quantité qui joue le rôle de nouvelle observation en apprentissage par différence temporelle.

Définition 2.5 (Cible TD à un pas). La **cible TD** (*TD target*) à un pas au temps t est la quantité

$$Y_t^{\text{TD}} = R_{t+1} + \gamma V(S_{t+1}).$$

Cette cible remplace, dans TD(0), le retour complet G_t utilisé par les méthodes Monte Carlo. Elle est disponible immédiatement après l'observation d'une seule transition.

Définition 2.6 (TD(0)). La méthode **TD(0)** met à jour l'estimation d'un état observé selon la règle

$$V(S_t) \leftarrow V(S_t) + \alpha(Y_t^{\text{TD}} - V(S_t)),$$

c'est-à-dire

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t,$$

où l'**erreur TD** (*TD error*) est

$$\delta_t = Y_t^{\text{TD}} - V(S_t) = R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

Cette écriture met en évidence une idée importante : la mise à jour TD(0) a la même forme générale qu'une moyenne incrémentale. Dans une moyenne incrémentale, une estimation courante est corrigée dans la direction de l'écart entre une nouvelle observation et l'estimation actuelle. Ici, l'estimation courante est $V(S_t)$, et la nouvelle "observation" n'est pas un retour complet, mais la cible TD

$$Y_t^{\text{TD}} = R_{t+1} + \gamma V(S_{t+1}).$$

Autrement dit, la mise à jour

$$V(S_t) \leftarrow V(S_t) + \alpha(Y_t^{\text{TD}} - V(S_t))$$

peut se lire ainsi : on déplace l'estimation actuelle $V(S_t)$ d'une petite quantité dans la direction de la cible observée à l'instant t .

Interprétons soigneusement cette formule.

- $V(S_t)$ est la valeur actuellement attribuée à l'état courant.
- $Y_t^{\text{TD}} = R_{t+1} + \gamma V(S_{t+1})$ est la cible TD à un pas, inspirée de l'équation de Bellman.
- La différence

$$\delta_t = Y_t^{\text{TD}} - V(S_t)$$

mesure l'erreur locale commise par l'estimation actuelle.

- La mise à jour corrige donc $V(S_t)$ dans le sens indiqué par cette erreur.

Si $\delta_t > 0$, la cible TD est plus grande que la valeur actuellement estimée : l'expérience récente suggère que l'état S_t vaut davantage que ce que l'on pensait, et l'on augmente donc $V(S_t)$. Si $\delta_t < 0$, la cible TD est plus petite que l'estimation actuelle : l'expérience récente suggère au contraire que la valeur de S_t a été surestimée, et l'on diminue donc $V(S_t)$.

Cette interprétation est très proche de celle de la moyenne incrémentale vue plus haut. Dans le cas Monte Carlo, la nouvelle information était le retour complet G_t et la mise à jour avait la forme

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t)).$$

Dans TD(0), on retrouve exactement la même structure, mais avec

$$G_t \text{ remplacé par } Y_t^{\text{TD}} = R_{t+1} + \gamma V(S_{t+1}).$$

Ainsi, Monte Carlo et TD(0) reposent tous deux sur une correction progressive d'une estimation courante par l'écart à une nouvelle cible ; la différence essentielle est que Monte Carlo utilise un retour complet, tandis que TD(0) utilise une cible bootstrapée à un pas.

2.2.3 Offline vs online

Il est utile de distinguer deux façons d'exécuter les mises à jour.

Une méthode est dite **offline** si elle collecte d'abord un ensemble de données, puis effectue les mises à jour après coup. C'est l'esprit naturel du Monte Carlo par épisodes complets.

Une méthode est dite **online** si elle met à jour les estimations au fur et à mesure des transitions observées. TD(0) est typiquement une méthode online : dès que la transition

$$S_t, A_t, R_{t+1}, S_{t+1}$$

est observée, on peut former la cible TD

$$Y_t^{\text{TD}} = R_{t+1} + \gamma V(S_{t+1})$$

et corriger immédiatement $V(S_t)$.

L'avantage principal du mode online est la réactivité. L'information n'attend pas la fin de l'épisode pour se propager. C'est particulièrement important lorsque les épisodes sont longs ou lorsque l'environnement ne possède pas de terminaison nette.

2.2.4 Généralisation à n -step TD(n)

Monte Carlo et TD(0) peuvent être vus comme les deux extrêmes d’une famille plus large. La généralisation naturelle de TD(0) consiste à considérer des cibles à plusieurs pas. Cette méthode, appelée TD(n), ou encore n -step TD, remplace la cible à un pas de TD(0) par une cible à n pas.

Pour un entier $n \geq 1$, on définit la cible à n pas

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

Cette formule mérite d’être lue lentement. On regarde d’abord n récompenses réellement observées, puis on *bootstrap* à partir de l’état atteint au bout de n pas. Lorsque $n = 1$, on retrouve TD(0) :

$$G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1}).$$

Lorsque n est égal à la durée restante de l’épisode, il n’y a plus de bootstrap final et l’on retombe sur le retour Monte Carlo complet.

Ainsi, les méthodes n -step forment un pont entre TD(0) et Monte Carlo.

- petit n : mises à jour rapides, davantage de bootstrap ;
- grand n : cibles plus proches du vrai retour, mais variance plus grande et mises à jour plus tardives.

L’intérêt de TD(n) est donc de permettre un compromis entre deux effets opposés. D’un côté, lorsque n est petit, la mise à jour peut être effectuée rapidement, mais elle dépend fortement de valeurs estimées, qui peuvent encore être imprécises au début de l’apprentissage. D’un autre côté, lorsque n est plus grand, la cible incorpore davantage de récompenses réellement observées, et le terme de bootstrap final

$$\gamma^n V(S_{t+n})$$

a une influence plus réduite lorsque $\gamma < 1$. On peut donc espérer, dans certaines situations, une cible plus informative que celle de TD(0), en particulier au début de l’apprentissage.

Il ne faut cependant pas en conclure qu’un grand n est toujours meilleur. Lorsque n augmente, il faut attendre davantage d’observations avant de pouvoir effectuer la mise à jour, et la cible peut devenir plus variable. Le choix de n dépend donc du problème étudié ; en pratique, c’est un hyperparamètre que l’on choisit souvent par expérimentation.

D’un point de vue algorithmique, TD(n) s’utilise naturellement avec une fenêtre glissante. Après avoir observé suffisamment de transitions, on peut mettre à jour $V(S_t)$ à partir des n pas suivants, puis, à l’observation suivante, mettre à jour $V(S_{t+1})$ à partir d’une nouvelle fenêtre décalée d’un cran. Les fenêtres successives se recouvrent donc généralement. On peut voir cela comme l’utilisation d’un petit tampon (*buffer*) de longueur n contenant les transitions les plus récentes.

2.2.5 Exemples classiques pour TD

Exemple 2.7 (Temps de trajet : Monte Carlo versus TD(0), Sutton et Barto [2], exemple 6.1). Considérons un exemple classique de prédiction par différence temporelle. Un conducteur rentre chez lui et traverse successivement plusieurs *situations* (que l’on peut lire ici comme des états) : quitter le bureau, atteindre la voiture, sortir de l’autoroute, se retrouver derrière un camion dans une rue secondaire, puis entrer dans la rue de la maison.

Cet exemple est essentiellement illustratif : il ne met pas en scène un problème de contrôle où l’agent chercherait à maximiser une récompense, mais un problème de prédiction. Le but est d’estimer, à chaque étape du trajet, le temps total nécessaire pour rentrer à la maison. À chaque nouvelle situation rencontrée, on dispose donc d’une estimation courante de ce temps total, qui peut être corrigée à la lumière des

observations. L'intérêt de l'exemple est précisément de montrer que Monte Carlo et TD(0) effectuent cette correction de manière différente : Monte Carlo corrige chaque prédiction à partir du résultat final effectivement observé, c'est-à-dire la durée totale réelle du trajet, tandis que TD(0) la corrige localement à partir de la prédiction de l'étape suivante.

À chaque situation, le conducteur possède une estimation du *temps restant* avant l'arrivée, ainsi qu'une estimation du *temps total* du trajet. On note :

$$\hat{\tau}(s) = \text{temps restant prédit à partir de l'état } s,$$

et

$$\hat{T}(s) = \text{temps total prédit pour le trajet si l'on se trouve dans l'état } s.$$

Ces deux quantités sont liées par

$$\hat{T}(s) = e(s) + \hat{\tau}(s),$$

où $e(s)$ désigne le temps déjà écoulé lorsque l'on atteint l'état s .

Dans l'épisode observé ci-dessous, le trajet effectif dure finalement 43 minutes.

État	Temps écoulé, $e(s)$ (minutes)	Temps restant prédit, $\hat{\tau}(s)$	Temps total prédit, $\hat{T}(s)$
quitter le bureau	0	30	30
atteindre la voiture	5	35	40
sortie d'autoroute	20	15	35
derrière un camion	30	10	40
rue de la maison	40	3	43
arrivée à la maison	43	0	43

Lecture Monte Carlo. Une méthode Monte Carlo attend la fin de l'épisode, puis compare toutes les prédictions antérieures au résultat final observé. Ici, le temps total effectivement observé est 43 minutes. Si l'on prend un pas d'apprentissage $\alpha = 1$, toutes les prédictions de temps total encore inexacts sont remplacées par 43 :

$$30 \rightarrow 43, \quad 40 \rightarrow 43, \quad 35 \rightarrow 43, \quad 40 \rightarrow 43, \quad 43 \rightarrow 43.$$

Autrement dit, Monte Carlo corrige chaque état visité à partir de l'*issue finale complète*.

Lecture TD(0). Une méthode TD(0), au contraire, n'attend pas l'issue finale pour mettre à jour une prédiction. Elle corrige localement une estimation à partir de ce que l'on vient d'observer et de l'estimation déjà disponible pour l'état suivant.

Si l'on travaille avec les *temps restants prédits*, la cible TD naturelle à un pas est

$$\Delta t_{t+1} + \hat{\tau}(S_{t+1}),$$

où Δt_{t+1} est le temps effectivement écoulé entre S_t et S_{t+1} . Mais, comme

$$\hat{T}(S_t) = e(S_t) + \hat{\tau}(S_t) \quad \text{et} \quad e(S_{t+1}) = e(S_t) + \Delta t_{t+1},$$

cela revient exactement, sur l'échelle des *temps totaux prédits*, à déplacer $\hat{T}(S_t)$ vers $\hat{T}(S_{t+1})$.

Avec $\alpha = 1$, on obtient donc :

$$30 \rightarrow 40, \quad 40 \rightarrow 35, \quad 35 \rightarrow 40, \quad 40 \rightarrow 43, \quad 43 \rightarrow 43.$$

On voit ainsi que TD ne force pas immédiatement toutes les prédictions à devenir égales au résultat final. L'information se propage *de proche en proche* le long de la trajectoire.

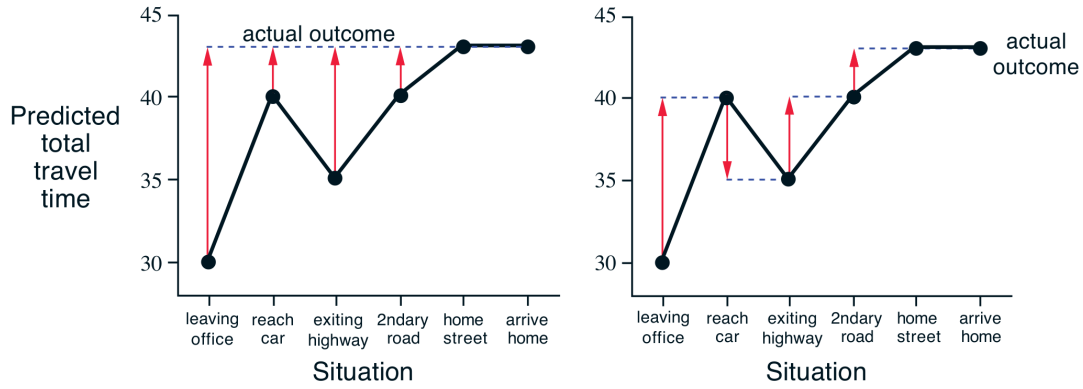
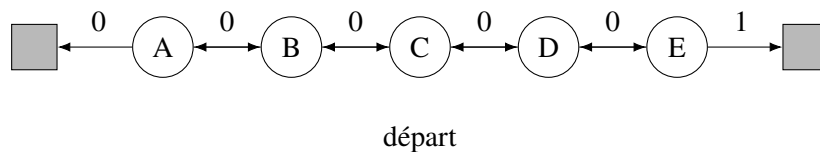


FIGURE 2.1 – Corrections recommandées dans l'exemple du retour à la maison par les méthodes de Monte Carlo (à gauche) et les méthodes TD (à droite). Figure reproduite à partir de [2], exemple 6.1, sous licence CC BY-NC-ND 2.0.

Cet exemple met bien en évidence la différence conceptuelle entre les deux approches. Monte Carlo utilise l'information globale donnée par la fin de l'épisode : toutes les prédictions sont corrigées vers l'issue finale observée. TD(0), au contraire, exploite immédiatement l'information disponible localement et impose une cohérence entre états successifs : chaque prédiction est corrigée vers celle de l'état suivant. L'information finale ne remonte donc pas instantanément jusqu'au début du trajet ; elle se propage progressivement d'une visite à l'autre, au fil des épisodes.

On peut se demander pourquoi l'on dit que Monte Carlo doit attendre la fin de l'épisode, alors que la moyenne empirique peut être mise à jour de manière incrémentale (Sec. 2.1.4). La formule de moyenne incrémentale ne supprime pas en effet l'attente de la fin de l'épisode en Monte Carlo ; elle permet seulement de mettre à jour l'estimation sans stocker tous les retours passés. En effet, la quantité à moyenner reste le retour complet G_t , qui n'est connu qu'une fois l'épisode terminé. La différence essentielle avec TD(0) n'est donc pas la possibilité d'une mise à jour incrémentale, mais le fait que la cible de TD(0) est disponible immédiatement après une seule transition.

Exemple 2.8 (Random Walk de Sutton et Barto [2], exemple 6.2). Chaîne aléatoire à états terminaux : chaîne d'états alignés de gauche à droite, avec deux états terminaux aux extrémités. Le but est d'évaluer la probabilité d'atteindre le terminal de droite, sous une marche aléatoire équiprobable.



Cet exemple classique compare expérimentalement deux méthodes de prédiction *model free* : TD(0) et Monte Carlo à pas constant (*constant-step-size MC*). Ici, « à pas constant » signifie que la mise à jour utilise un coefficient d'apprentissage fixe α , par exemple

$$V(s) \leftarrow V(s) + \alpha(G_t - V(s)),$$

au lieu du coefficient $\frac{1}{N(s)}$ de la moyenne empirique. Chaque nouveau retour conserve ainsi un poids comparable au cours de l'apprentissage. Le cadre est volontairement très simple, afin de faire apparaître clairement la différence entre les deux approches. Les méthodes TD montrent comment l'information peut se propager à partir de transitions locales : Une transition vers un état légèrement meilleur fait déjà monter la valeur de l'état courant, même sans attendre la fin complète de l'épisode.

On considère ici un *Markov reward process* (MRP), c'est-à-dire un processus de décision de Markov

sans choix d'action. Il n'y a donc pas de politique à optimiser : le seul objectif est d'estimer correctement la fonction de valeur des états.

Les états non terminaux sont

$$A, B, C, D, E,$$

auxquels on ajoute deux états terminaux, l'un à gauche et l'autre à droite. Chaque épisode commence en C . À chaque pas, le processus se déplace avec probabilité $\frac{1}{2}$ vers la gauche et avec probabilité $\frac{1}{2}$ vers la droite.

Tous les gains sont nuls, sauf lors d'une terminaison à droite, qui produit une récompense de $+1$. La terminaison à gauche produit une récompense nulle. Le problème est *non actualisé* ($\gamma = 1$).

Comme le seul gain non nul est obtenu si l'on termine à droite, la valeur d'un état est simplement la probabilité d'atteindre l'état terminal droit en partant de cet état. On obtient ainsi les vraies valeurs

$$v(A) = \frac{1}{6}, \quad v(B) = \frac{2}{6}, \quad v(C) = \frac{3}{6} = \frac{1}{2}, \quad v(D) = \frac{4}{6}, \quad v(E) = \frac{5}{6}.$$

Cet exemple est intéressant car, malgré sa simplicité, il met en évidence une différence importante entre TD(0) et Monte Carlo.

- La méthode **Monte Carlo** attend la fin complète de l'épisode pour former une cible de mise à jour.
- La méthode **TD(0)** met à jour dès qu'une transition est observée, en utilisant une cible bootstrapée à un pas.

Autrement dit, Monte Carlo apprend à partir du *résultat final complet*, tandis que TD(0) apprend *de proche en proche*, en imposant progressivement une cohérence entre états voisins.

Dans l'expérience de Sutton et Barto, toutes les estimations sont initialisées à

$$V(s) = 0.5 \quad \text{pour tout } s \in \{A, B, C, D, E\}.$$

Cette valeur initiale est intermédiaire : elle n'est ni absurde ni exacte. L'intérêt de l'expérience est précisément d'observer comment les méthodes corrigent peu à peu cette initialisation.

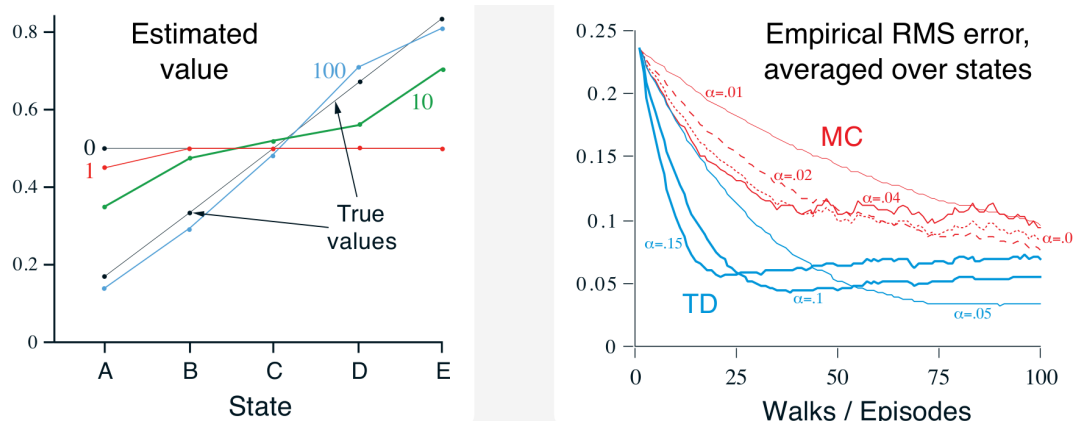


FIGURE 2.2 – Chaîne aléatoire à états terminaux. A gauche : estimations TD apprises après plusieurs épisodes. A droite : l'erreur quadratique moyenne (MC vs TD). Exemple adaptée de Sutton et Barto [2, ex. 6.2].

La Figure 2.2 comporte deux panneaux. Le message principal de l'exemple apparaît surtout dans le panneau de droite : pour cette tâche de prédiction, TD(0) converge plus rapidement que Monte Carlo, comme le montre la décroissance plus rapide de l'erreur RMS.

Panneau de gauche. Il montre, pour TD(0), les estimations apprises après un petit nombre d'épisodes (1 : rouge, 10 : vert, 100 : blue). Au début, les valeurs restent proches de l'initialisation 0.5, puis elles

se déforment progressivement pour se rapprocher des vraies valeurs $\frac{1}{6}, \frac{2}{6}, \frac{3}{6}, \frac{4}{6}, \frac{5}{6}$. On voit donc concrètement l'idée déjà rencontrée plus haut : l'apprentissage consiste à corriger progressivement une table de valeurs initialement arbitraire.

Panneau de droite. Il montre l'erreur RMS empirique (*root mean squared error*), c'est-à-dire l'écart quadratique moyen entre les valeurs apprises et les vraies valeurs, moyenné sur les états puis sur un grand nombre d'exécutions. Les courbes bleues correspondent à TD(0), les courbes rouges à Monte Carlo, pour plusieurs valeurs du pas d'apprentissage α . On observe que, dans cet exemple, les courbes TD descendent plus vite et plus bas que les courbes Monte Carlo : TD(0) est donc plus efficace sur cette tâche de prédiction.

Pourquoi TD(0) est-il avantage ici ? Dans cette marche aléatoire, l'information utile est très locale : les états voisins ont des valeurs proches, et la structure du problème permet à l'information de se propager rapidement d'un état à l'autre. Une mise à jour bootstrapée à un pas est donc particulièrement bien adaptée. Monte Carlo, au contraire, attend la fin de l'épisode et n'exploite l'information qu'avec retard.

Cet exemple illustre donc une idée importante : même lorsque deux méthodes convergent en principe vers la même fonction de valeur, leur comportement pratique pendant l'apprentissage peut être très différent. TD(0) peut apprendre plus vite parce qu'il met à jour immédiatement les estimations et propage l'information au fil même de la trajectoire observée.

Exemple 2.9 (Prédire les retours : batch Monte Carlo vs batch TD(0), Sutton et Barto [2], exemple 6.4). Considérons maintenant un tout petit processus de récompense markovien inconnu, dans lequel on observe simplement les huit épisodes suivants :

$A, 0, B, 0$	$B, 1$
$B, 1$	$B, 1$
$B, 1$	$B, 1$
$B, 1$	$B, 0$

Le premier épisode signifie : on part de l'état A , on passe à l'état B avec une récompense nulle, puis l'épisode se termine depuis B avec une récompense nulle. Les sept autres épisodes commencent directement en B et se terminent immédiatement, avec une récompense soit égale à 1, soit égale à 0.

La question est la suivante : quelles valeurs faut-il attribuer à $V(A)$ et $V(B)$ à partir de ces seules données ?

Commençons par l'état B . Parmi les huit épisodes observés, l'état B est visité huit fois au total :

- six fois, le retour observé depuis B vaut 1 ;
- deux fois, le retour observé depuis B vaut 0.

Il est donc naturel d'estimer

$$V(B) = \frac{6}{8} = \frac{3}{4}.$$

La difficulté porte alors sur la valeur de A .

Réponse de type Monte Carlo. L'état A n'apparaît qu'une seule fois dans les données, au début du premier épisode, et le retour complet observé depuis A dans cet épisode vaut

$$G = 0 + 0 = 0.$$

Une méthode Monte Carlo par lots (*batch Monte Carlo*) conduit donc naturellement à poser

$$V(A) = 0.$$

En effet, Monte Carlo cherche à ajuster les estimations aux retours *effectivement observés* dans les épisodes de l'échantillon. Ici, pour A , il n'y a qu'une seule observation, et cette observation vaut 0.

Réponse de type TD(0). Une méthode TD(0) par lots (*batch TD(0)*) raisonne différemment. Dans toutes les données observées, chaque fois que l'on est en A , on passe immédiatement en B avec une récompense nulle. La relation de Bellman empirique suggère donc

$$V(A) \approx 0 + V(B) = V(B).$$

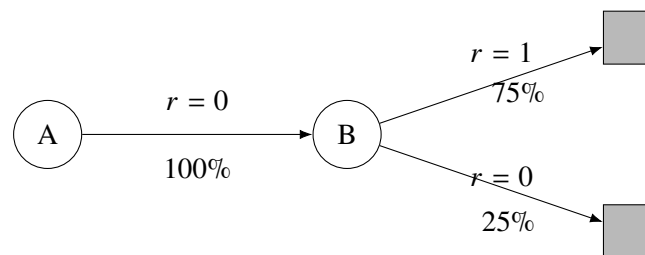
Comme on a déjà estimé

$$V(B) = \frac{3}{4},$$

on obtient alors

$$V(A) = \frac{3}{4}.$$

Autrement dit, TD(0) exploite ici la structure de transition observée : comme A mène systématiquement vers B avec une récompense nulle, la valeur de A doit être très proche de celle de B .



La figure résume le modèle empirique suggéré par les données :

- depuis A , on va toujours vers B avec une récompense nulle ;
- depuis B , on termine avec récompense 1 dans 75% des cas et avec récompense 0 dans 25% des cas.

Dans ce petit exemple, les deux approches donnent donc deux réponses différentes :

$$\text{batch Monte Carlo : } V(A) = 0, \quad \text{batch TD(0) : } V(A) = \frac{3}{4}.$$

Interprétation. La réponse Monte Carlo colle parfaitement aux retours observés dans l'échantillon : pour l'unique visite de A , le retour observé est effectivement nul. Elle minimise donc naturellement l'erreur sur ces données d'entraînement.

La réponse TD(0), elle, ne se contente pas de mémoriser les retours observés. Elle cherche aussi à rendre les estimations cohérentes avec les transitions observées entre états. Si le processus est bien markovien, cette contrainte supplémentaire peut conduire à une meilleure capacité de généralisation : même si l'on n'a observé qu'une seule fois l'état A , le fait qu'il mène systématiquement à B apporte une information importante sur sa valeur.

Cet exemple montre bien une différence profonde entre les deux familles de méthodes :

- Monte Carlo apprend directement à partir des retours observés ;
- TD apprend en combinant les observations et la structure des transitions.

Exemple 2.10 (Grid world avec récompense terminale). Dans un petit monde quadrillé, supposons une récompense nulle partout sauf +1 à l'arrivée dans l'état terminal favorable. Si un état voisin du but reçoit une forte valeur estimée, TD(0) transmet progressivement cette information aux états plus éloignés. On retrouve l'intuition de propagation locale déjà présente dans value iteration, mais ici cette propagation se fait à partir de transitions observées plutôt qu'à partir du modèle exact.

Exemple 2.11 (Prédiction en backgammon). Un succès historique marquant des méthodes de différence temporelle est TD-Gammon, le programme de Gerald Tesauro pour le backgammon [5]. L'idée générale est d'apprendre une fonction d'évaluation des positions : après une transition observée pendant une partie, la valeur estimée d'une position est corrigée à partir de la valeur estimée de la position suivante. Cet exemple est devenu célèbre parce qu'il montre qu'un signal terminal rare – gagner ou perdre la partie – peut néanmoins être propagé progressivement de position en position par apprentissage TD.

2.2.6 Monte Carlo versus TD(0)

Le contraste entre les deux méthodes est important.

Monte Carlo	TD(0)
Utilise un retour complet	Utilise une cible à un pas
Attend en général la fin de l'épisode	Mise à jour immédiate
Pas de bootstrap	Bootstrap sur $V(S_{t+1})$
Variance souvent plus grande	Biais supplémentaire possible, mais variance souvent plus faible

On dit souvent, de manière un peu informelle mais utile, que Monte Carlo est plus *fidèle* au vrai retour observé, tandis que TD est plus *économe* et plus *réactif*. En pratique, TD se révèle souvent plus efficace lorsqu'on apprend en ligne à partir de nombreuses transitions.

2.3 TD(λ)

2.3.1 Transition depuis le n -step TD

Les méthodes TD à plusieurs pas permettent déjà d'aller au-delà de TD(0). Au lieu de regarder seulement une récompense puis de *bootstrapper*, on regarde plusieurs récompenses successives avant de faire intervenir la valeur d'un état futur. Cela conduit aux cibles à n pas :

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

Cette idée est très naturelle, mais elle soulève immédiatement une question : *quelle valeur de n faut-il choisir ?*

- Si n est petit, la mise à jour est rapide et locale, mais elle dépend fortement du *bootstrap*.
- Si n est grand, on se rapproche davantage du vrai retour observé, mais il faut attendre plus longtemps et la variance peut augmenter.

Autrement dit, il n'existe pas toujours une valeur de n qui soit universellement meilleure que les autres. Une idée naturelle est alors de ne pas choisir une seule profondeur temporelle, mais de combiner plusieurs cibles à plusieurs pas.

C'est précisément l'idée de **TD(λ)**.

2.3.2 Idée générale

La méthode **TD(λ)** cherche à combiner les avantages de TD(0) et des méthodes à plusieurs pas. Le paramètre $\lambda \in [0, 1]$ règle l'équilibre entre mises à jour locales rapides et prise en compte de portions plus longues du futur.

Au lieu de choisir une seule cible (par exemple à un pas, ou à deux pas, ou à durée complète) on mélange plusieurs cibles de longueurs différentes. Les cibles courtes propagent vite l'information ; les cibles longues ressemblent davantage au vrai retour.

Pour cela, on introduit le **retour** λ (λ -return), défini comme une moyenne pondérée des retours à plusieurs pas :

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}.$$

Cette formule mérite une lecture attentive. Le terme $G_t^{(1)}$ est la cible de TD(0), le terme $G_t^{(2)}$ est la cible à deux pas, puis viennent les cibles à trois pas, à quatre pas, etc. La quantité G_t^λ combine donc toutes ces approximations du retour en leur donnant des poids décroissants.

Plus précisément :

- $G_t^{(n)}$ désigne le retour à n pas ;
- le facteur $(1 - \lambda)\lambda^{n-1}$ attribue un poids aux différentes profondeurs temporelles ;
- lorsque λ est petit, les petites valeurs de n dominent ;
- lorsque λ est proche de 1, les retours plus longs prennent davantage d'importance.

Ainsi, TD(λ) forme une famille continue de méthodes intermédiaires entre TD(0) et Monte Carlo.

Lorsque $\lambda = 0$, tout le poids porte sur le retour à un pas, et l'on retrouve essentiellement TD(0). Lorsque λ est proche de 1, on se rapproche d'un comportement de type Monte Carlo dans les tâches épisodiques.

Intuition sur la valeur du λ . Lorsque λ est petit, l'apprentissage est plus **local** : les mises à jour s'appuient surtout sur des informations proches dans le temps, avec beaucoup de *bootstrap*. L'information se propage rapidement, mais de manière plus progressive.

Lorsque λ est grand, l'apprentissage devient plus **global** : les mises à jour tiennent compte de portions plus longues du futur, et se rapprochent davantage du retour réellement observé. L'information est alors propagée sur un horizon plus long, mais au prix d'une plus grande dépendance à l'épisode complet.

On peut donc voir λ comme un réglage de la *profondeur temporelle* de l'apprentissage : petit λ signifie “corriger à courte portée”, grand λ signifie “redistribuer le crédit sur une histoire plus longue”.

2.3.3 Vue forward : combinaison de tous les retours à plusieurs pas

Dans la **vue forward** (*forward view*), la mise à jour s'écrit conceptuellement

$$V_\pi(S_t) \leftarrow V_\pi(S_t) + \alpha(G_t^\lambda - V_\pi(S_t)).$$

Cette écriture est très naturelle : on compare l'estimation actuelle $V_\pi(S_t)$ à une cible plus riche, le retour λ , qui combine toutes les cibles à plusieurs pas.

On peut voir TD(λ) comme une réponse élégante à la question précédente : au lieu de trancher entre une cible à un pas, à deux pas, à trois pas ou plus, on les utilise toutes à la fois.

Cependant, cette présentation a une limite pratique importante. Pour calculer exactement G_t^λ , il faut en général connaître une grande partie du futur, souvent jusqu'à la fin de l'épisode. Dans cette forme, TD(λ) fonctionne donc plutôt comme une méthode **offline** : il faut disposer d'un épisode complet, ou au moins d'un long segment, avant d'effectuer la mise à jour exacte.

2.3.4 Vue *backward* et traces d'éligibilité

L'idée essentielle de la vue backward est que l'on regarde vers le passé récent pour répartir l'erreur TD courante. On ne met donc pas à jour tous les états indistinctement, mais seulement ceux qui ont été visités suffisamment récemment pour conserver une importance significative. Plus cette importance est grande, plus l'état reçoit une correction importante. On retrouve ainsi une intuition de propagation du crédit vers le passé : une information positive ou négative observée maintenant est attribuée, avec des poids décroissants, aux états récents qui ont conduit à la situation présente.

L'autre manière classique de comprendre $TD(\lambda)$ repose sur les **traces d'éligibilité** (*eligibility traces*). L'idée est d'associer à chaque état s un coefficient, noté par exemple $e_t(s)$, qui mesure à quel point cet état est encore « éligible » pour recevoir une partie des corrections au temps t .

Intuitivement, $e_t(s)$ joue le rôle d'une mémoire du passé récent :

- lorsqu'un état s est visité, sa trace augmente ;
- lorsqu'il n'est plus visité, sa trace décroît progressivement au cours du temps ;
- plus la trace d'un état est grande, plus cet état recevra une part importante de la correction courante.

Dans la version la plus simple, après chaque pas de temps, les traces décroissent d'un facteur voisin de $\gamma\lambda$. Le paramètre $\lambda \in [0, 1]$ contrôle donc la durée de cette mémoire :

- si $\lambda = 0$, les traces disparaissent immédiatement, et seule l'estimation de l'état courant est corrigée : on retrouve l'esprit de $TD(0)$;
- si λ est proche de 1, les traces persistent plus longtemps, et l'erreur TD courante peut être redistribuée à plusieurs états visités dans le passé récent.

Lorsque l'on observe l'erreur TD

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t),$$

on ne l'utilise donc pas seulement pour corriger $V(S_t)$, mais aussi, avec des poids donnés par les traces d'éligibilité, pour corriger certains états antérieurement visités. Les états les plus récents reçoivent en général la plus grande correction ; les états plus anciens reçoivent une correction plus faible :

$$V(s) \leftarrow V(s) + \alpha \delta_t e_t(s).$$

Cette formulation est appelée la **vue backward** (*backward view*). Elle n'utilise pas explicitement le retour G_t^λ , mais elle fournit une mise en oeuvre **online** très naturelle de la même idée générale. Autrement dit :

- la vue forward explique bien la signification mathématique de $TD(\lambda)$, comme moyenne pondérée de cibles à plusieurs horizons ;
- la vue backward explique comment propager, pas à pas, l'information vers le passé récent pendant l'interaction avec l'environnement.

On peut donc voir λ comme un paramètre de « profondeur du crédit » : plus λ est grand, plus une conséquence observée maintenant peut être attribuée à une chaîne longue d'états antérieurs ; plus λ est petit, plus la correction reste locale.

Nous n'entrons pas ici dans la définition complète des traces d'éligibilité ni dans les différentes variantes possibles de $TD(\lambda)$, car cela nous entraînerait trop loin pour le niveau visé dans ces notes. Il suffit de retenir que le paramètre λ intervient dans la manière dont les traces $e_t(s)$ mémorisent les états récemment visités et répartissent l'erreur TD vers le passé. Le lecteur intéressé trouvera une présentation détaillée dans Sutton et Barto [2, chap. 12].

Les points essentiels à garder après la lecture de ce chapitre sont :

- $TD(\lambda)$ prolonge naturellement les méthodes à plusieurs pas.
- Au lieu de choisir une seule cible à n pas, on combine toutes les approximations du retour.

- Le paramètre λ règle la profondeur temporelle prise en compte dans les mises à jour.
- Lorsque $\lambda = 0$, on retrouve essentiellement TD(0).
- Lorsque λ se rapproche de 1, on se rapproche de Monte Carlo dans les tâches épisodiques.
- La vue forward est conceptuellement simple mais demande en général des épisodes complets.
- La vue backward introduit les traces d'éligibilité et permet une mise en oeuvre online.

Chapitre 3

Contrôle Model Free

3.1 Introduction : contrôle versus prédiction

Dans un problème de **prédiction**, la politique π est fixée et l'on cherche seulement à estimer v_π ou q_π . Dans un problème de **contrôle**, la politique n'est plus donnée une fois pour toutes. On veut la faire évoluer de façon à améliorer la performance à long terme.

Cette distinction est fondamentale. En prédiction, les données sont produites par une politique imposée. En contrôle, les données sont produites par une politique qui est elle-même en train d'être modifiée. L'algorithme doit donc apprendre tout en changeant sa façon d'explorer.

3.1.1 Generalized Policy Iteration

Le concept unificateur ici est celui de **Generalized Policy Iteration** (GPI). Il s'agit de l'idée générale selon laquelle on alterne deux tendances :

- une tendance d'*évaluation*, qui rend les valeurs plus cohérentes avec la politique courante ;
- une tendance d'*amélioration*, qui rend la politique plus gloutonne par rapport aux valeurs courantes.

Dans la programmation dynamique, présentée dans le premier cours, cette alternance apparaissait déjà de manière exacte dans policy iteration [1, section 3.3]. En model-free, l'idée reste la même, mais les deux étapes sont approximatives et s'effectuent à partir de données échantillonnées.

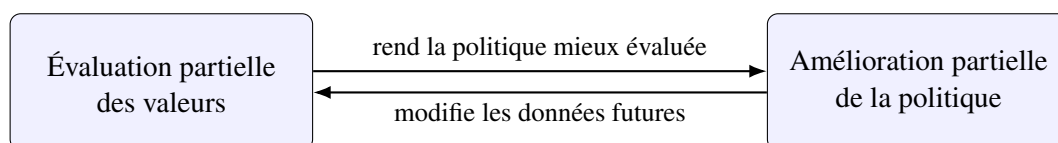


FIGURE 3.1 – Idée générale de la *generalized policy iteration*.

3.1.2 Politique ε -gloutonne et GLIE

Une politique purement gloutonne choisirait toujours l'action de plus grande valeur estimée. Ce serait naturel pour exploiter l'information acquise, mais dangereux pour l'apprentissage : une action sous-estimée au début pourrait ne jamais être réessayée.

On introduit donc la notion de politique **ε -gloutonne** (ε -greedy policy). Dans un état s ,

- avec probabilité $1 - \varepsilon$, on choisit une action maximisant $Q(s, a)$;
- avec probabilité ε , on choisit une action au hasard dans l'ensemble des actions disponibles.

Cette règle très simple réalise un compromis entre exploitation et exploration. Quand ε est grand, on explore davantage ; quand ε est petit, on exploite surtout ce que l'on croit savoir.

La condition **GLIE** (*Greedy in the Limit with Infinite Exploration*) signifie, de manière informelle, deux choses en même temps :

- toutes les actions pertinentes continuent d'être explorées indéfiniment ;
- la politique devient asymptotiquement gloutonne.

Typiquement, on choisit une suite $\varepsilon_t \downarrow 0$ lentement. Ainsi, au début l'agent explore beaucoup, puis il devient progressivement plus déterministe et plus exploitant.

3.1.3 GLIE Monte Carlo

Le contrôle Monte Carlo le plus simple consiste à itérer le schéma suivant :

1. générer des épisodes à l'aide d'une politique exploratoire, souvent ε -gloutonne ;
2. estimer $q_\pi(s, a)$ par moyenne des retours observés après les visites de (s, a) ;
3. améliorer la politique en la rendant ε -gloutonne par rapport aux valeurs estimées, où le paramètre d'exploration ε diminue progressivement avec le nombre d'itérations, de sorte que la politique devienne gloutonne à la limite tout en continuant à explorer suffisamment longtemps.

Le qualificatif GLIE ajoute l'idée que l'exploration décroît, mais ne disparaît pas trop vite. Cette stratégie permet d'espérer, sous des hypothèses adaptées, apprendre une politique optimale sans imposer d'*exploring starts* artificiels.

En pratique, on utilise toutefois parfois un paramètre ε constant pendant tout ou partie de l'apprentissage, afin de conserver une exploration simple et robuste ; on perd alors le cadre théorique strict de GLIE, mais on gagne souvent en simplicité de mise en œuvre.

3.2 On-policy et SARSA

3.2.1 Le problème on-policy

Avant de définir une méthode *on-policy*, il faut distinguer deux rôles possibles pour une politique.

La **politique de comportement** (*behavior policy*) est la politique qui sert effectivement à générer les données, c'est-à-dire celle qui choisit les actions pendant l'interaction avec l'environnement. C'est elle qui produit les trajectoires observées

$$(S_0, A_0, R_1, S_1, A_1, R_2, \dots).$$

La **politique cible** (*target policy*) est la politique dont on cherche à apprendre la valeur. Autrement dit, c'est la politique que l'on souhaite évaluer ou améliorer.

Ces deux politiques peuvent coïncider, mais ce n'est pas obligatoire. Lorsqu'elles sont identiques, on parle de méthode **on-policy**. Lorsqu'elles sont différentes, on parle de méthode **off-policy**.

Une méthode est donc **on-policy** lorsqu'elle apprend la valeur de la politique même qui sert à générer les données. Autrement dit, la politique de comportement et la politique cible coïncident.

Dans un cadre de contrôle, cela signifie que l'on évalue et améliore simultanément une politique exploratoire, par exemple une politique ε -gloutonne. On n'essaie pas d'apprendre la valeur d'une politique idéale totalement différente de celle que l'on exécute ; on apprend la valeur du comportement réel, exploration comprise.

C'est précisément l'esprit de **SARSA**. Le nom vient de la structure des données utilisées dans la mise à jour :

$$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}).$$

3.2.2 Algorithme SARSA

Définition 3.1 (SARSA). La méthode **SARSA** met à jour la valeur d'état-action selon

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \delta_t,$$

avec

$$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t),$$

où l'action A_{t+1} est effectivement choisie par la politique courante.

Cette formule est l'analogie en contrôle de TD(0), mais appliqué à la fonction q_π . La ressemblance avec l'équation de Bellman pour q_π du premier document est directe [1, section 2.10]. La différence est qu'au lieu de calculer une espérance exacte sur les états suivants et les actions suivantes, on utilise ici un seul échantillon effectivement observé.

3.2.3 Pourquoi SARSA est-il on-policy ?

Parce que la cible de mise à jour utilise l'action A_{t+1} réellement choisie par la politique actuelle. Si cette politique est ε -gloutonne, alors l'estimation Q apprend les conséquences à long terme de cette politique ε -gloutonne elle-même, y compris le fait qu'elle explore encore un peu.

C'est un point conceptuellement très important. SARSA ne demande pas :

“Que vaudrait cet état si l'on devenait parfaitement glouton à partir du prochain pas ?”

Il demande plutôt :

“Que vaut cet état-action si l'on continue à agir selon la politique actuelle, avec son mélange d'exploitation et d'exploration ?”

3.2.4 Garanties mathématiques : idée générale

Sous des hypothèses classiques (espaces finis, visites infinies des couples (s, a) , pas d'apprentissage vérifiant les conditions de Robbins–Monro¹, politique GLIE) on peut montrer que SARSA converge vers q_* avec probabilité 1.

À ce niveau, il n'est pas utile de détailler toute la preuve, mais il est utile d'en comprendre l'esprit.

- Les couples (s, a) sont visités un nombre infini de fois, donc l'algorithme continue d'apprendre partout où c'est nécessaire.
- Les pas d'apprentissage décroissent suffisamment pour stabiliser l'estimation, sans décroître trop vite afin de laisser l'information nouvelle jouer un rôle.
- La politique devient de plus en plus gloutonne, tout en continuant assez longtemps l'exploration.

1. Les conditions de Robbins–Monro désignent typiquement des pas d'apprentissage (α_t) tels que $\sum_t \alpha_t = +\infty$ et $\sum_t \alpha_t^2 < +\infty$. Elles jouent un rôle central dans l'analyse de convergence des méthodes d'approximation stochastique.

3.2.5 Extensions de SARSA

Plusieurs extensions apparaissent naturellement.

- **Expected SARSA** remplace $Q(S_{t+1}, A_{t+1})$ par l'espérance, sous la politique courante, des valeurs possibles dans l'état suivant :

$$\sum_a \pi(a | S_{t+1}) Q(S_{t+1}, a).$$

Cela réduit souvent la variance de la cible.

- **n -step SARSA** utilise des cibles à plusieurs pas, exactement dans l'esprit du passage de TD(0) à n -step TD.
- **SARSA(λ)** ajoute des traces d'éligibilité pour redistribuer l'erreur sur plusieurs couples état-action récents.

3.2.6 Similitudes avec TD(0)

SARSA est à TD(0) ce que la valeur d'état-action est à la valeur d'état.

- TD(0) :

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t)).$$

- SARSA :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)).$$

La structure est exactement la même : cible à un pas, bootstrap, correction proportionnelle à une erreur TD.

3.3 Off-policy et Q-Learning

3.3.1 Off-policy learning

Une méthode est dite **off-policy** lorsqu'elle apprend la valeur d'une politique cible μ à partir de données générées par une autre politique b . Cette séparation est très puissante. Elle permet par exemple d'explorer avec une politique encore prudente ou aléatoire tout en apprenant déjà la valeur d'une politique plus ambitieuse.

Conceptuellement, cela revient à dissocier deux rôles :

- la politique de comportement μ , qui collecte les données ;
- la politique cible π , que l'on cherche à évaluer ou à optimiser.

Cette dissociation est au coeur de Q-learning.

3.3.2 Importance sampling

Supposons d'abord que l'on veuille calculer une espérance sous une distribution P , alors que les échantillons disponibles proviennent d'une autre distribution Q . Dans le cas discret,

$$\mathbb{E}_P[f(X)] = \sum_x P(x) f(x) = \sum_x Q(x) \frac{P(x)}{Q(x)} f(x) = \mathbb{E}_Q \left[\frac{P(X)}{Q(X)} f(X) \right],$$

à condition que

$$Q(x) > 0 \quad \text{dès que} \quad P(x) > 0.$$

Le facteur

$$\frac{P(X)}{Q(X)}$$

s'appelle **poids d'importance** (*importance weight*). Il sert à corriger le fait que les échantillons n'ont pas été générés sous la bonne distribution.

Dans le cadre off-policy, le principe est exactement le même. On veut évaluer une politique cible π , mais les trajectoires ont été générées par une politique de comportement μ . La différence entre les deux distributions vient donc uniquement des probabilités d'action choisies par π et μ ; la dynamique de l'environnement, elle, est la même et se simplifie dans le rapport.

Pour une portion d'épisode allant du temps t jusqu'à la fin T , on définit ainsi le poids d'importance

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{\mu(A_k | S_k)}.$$

Ce poids n'est bien défini que si

$$\mu(a | s) > 0 \quad \text{dès que} \quad \pi(a | s) > 0,$$

c'est-à-dire si la politique de comportement explore suffisamment pour ne jamais rendre impossible une action que la politique cible pourrait choisir.

L'intuition est simple. Si la suite d'actions observée serait plus probable sous π que sous μ , alors son poids est supérieur à 1 et cette trajectoire doit compter davantage. Si elle serait moins probable sous π , son poids est inférieur à 1 et son influence doit être réduite.

3.3.3 Off-policy Monte Carlo

En Monte Carlo off-policy, on veut utiliser des épisodes générés par μ pour estimer la valeur de la politique cible π .

Pour la valeur d'un état, l'idée est de corriger le retour complet par le poids d'importance sur toute la fin de l'épisode :

$$v_{\pi}(s) = \mathbb{E}_{\mu}[\rho_{t:T-1} G_t | S_t = s].$$

Autrement dit, sous la politique de comportement μ , le retour brut G_t n'a pas en général la bonne moyenne pour estimer $v_{\pi}(s)$; mais le retour pondéré

$$\rho_{t:T-1} G_t$$

corrige cet écart.

On peut alors construire des estimateurs de type Monte Carlo. Par exemple, si l'on observe plusieurs visites de l'état s , on peut utiliser l'estimateur ordinaire

$$V(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} \rho_i G_i,$$

ou bien l'estimateur *pondéré* (ou *normalisé*)

$$V(s) \approx \frac{\sum_{i=1}^{N(s)} \rho_i G_i}{\sum_{i=1}^{N(s)} \rho_i}.$$

Le second est souvent plus stable en pratique, au prix d'un léger biais à taille d'échantillon finie.

L'idée est élégante, mais elle a une faiblesse importante : le poids

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{\mu(A_k | S_k)}$$

est un produit sur toute la suite future des actions. Si l'épisode est long, ce produit peut devenir extrêmement grand ou extrêmement petit. La variance des estimateurs peut alors exploser. C'est la principale difficulté des méthodes Monte Carlo off-policy.

3.3.4 Off-policy TD

Les idées off-policy peuvent aussi être combinées avec des mises à jour de type TD. Dans le cas de TD(0), on remplace le retour complet par une cible à un pas, puis on corrige la différence entre politique cible et politique de comportement par un facteur d'importance à un pas :

$$\rho_t = \frac{\pi(A_t | S_t)}{\mu(A_t | S_t)}.$$

La mise à jour prend alors la forme

$$V(S_t) \leftarrow V(S_t) + \alpha \rho_t (R_{t+1} + \gamma V(S_{t+1}) - V(S_t)).$$

Cette formule mérite une lecture attentive. Le terme

$$R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

est l'erreur TD habituelle, et le facteur ρ_t corrige le fait que l'action A_t a été choisie selon μ alors que l'on cherche à apprendre la valeur sous π .

L'intuition est la suivante. En Monte Carlo off-policy, il faut corriger toute la suite future des actions, car le retour G_t dépend de l'épisode complet. En TD(0), on ne regarde qu'une cible à un pas ; le futur plus lointain n'apparaît plus explicitement sous forme de trajectoire, mais est résumé par l'estimation $V(S_{t+1})$. Il suffit donc, à ce niveau élémentaire, de corriger le choix de l'action courante.

Cette approche est souvent plus réactive que Monte Carlo, mais elle reste délicate lorsque π et μ diffèrent beaucoup. Les rapports

$$\frac{\pi(A_t | S_t)}{\mu(A_t | S_t)}$$

peuvent devenir grands, ce qui augmente la variance et peut nuire à la stabilité. C'est l'une des raisons pour lesquelles les méthodes off-policy les plus utilisées en contrôle, comme Q-learning, exploitent une structure particulière qui évite d'écrire explicitement un produit complet de poids sur tout l'avenir.

3.3.5 Q-Learning

La difficulté des méthodes off-policy générales est donc claire : dès que l'on cherche à corriger explicitement l'écart entre la politique cible π et la politique de comportement μ , des rapports de probabilités apparaissent, et ces facteurs peuvent rendre l'apprentissage instable.

C'est ici qu'intervient **Q-learning**. L'idée décisive est la suivante : dans un problème de contrôle, on ne cherche pas seulement à évaluer une politique fixée, mais à approcher la fonction de valeur optimale d'action q_* . Or q_* vérifie l'équation de Bellman optimale

$$q_*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a \right].$$

Cette équation ne fait pas intervenir une action future effectivement tirée selon une politique donnée ; elle fait intervenir directement la *meilleure* action possible au pas suivant, à travers l'opérateur max.

C'est ce point qui change tout. Dans une méthode off-policy Monte Carlo, ou dans une méthode off-policy TD générale, il faut corriger le fait que les actions futures observées ont été choisies selon la politique de comportement μ alors que l'on veut raisonner sous une autre politique cible π . D'où les facteurs d'importance. En Q-learning, au contraire, la cible à un pas

$$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$$

ne dépend pas de l'action réellement choisie ensuite par l'agent. Elle dépend seulement de l'état suivant S_{t+1} et de la meilleure valeur actuellement estimée dans cet état.

Autrement dit, les données sont bien générées par une politique de comportement μ , par exemple une politique ε -gloutonne exploratoire, mais la quantité apprise correspond à une politique cible gloutonne. Q-learning est donc bien une méthode **off-policy** : on apprend la valeur d'une politique différente de celle qui génère effectivement les données. Cependant, comme la cible utilise directement un maximum au lieu de suivre les actions réellement observées dans le futur, il n'est pas nécessaire d'écrire un produit explicite de poids d'importance sur toute la suite des actions.

On peut résumer la situation ainsi :

- la politique de comportement μ sert à explorer et à produire les transitions ;
- la politique cible est la politique gloutonne associée aux valeurs Q ;
- la mise à jour n'utilise pas l'action future effectivement échantillonnée, mais la meilleure action selon l'estimation courante ;
- c'est cette substitution qui donne à Q-learning son caractère off-policy sans correction explicite par importance sampling sur tout l'avenir.

Cela conduit à la règle de mise à jour fondamentale de Q-learning.

Définition 3.2 (Q-learning). La méthode **Q-learning** met à jour la valeur d'état-action selon

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \delta_t,$$

avec

$$\delta_t = R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t).$$

La formule est la version échantillonnée de l'équation de Bellman optimale pour q_* [1, section 2.10]. En remplaçant l'espérance exacte sur les états suivants par la transition observée et en gardant le maximum sur les actions futures, on obtient précisément la mise à jour de Q-learning.

La différence avec SARSA est très nette : la cible n'utilise pas l'action effectivement choisie au temps suivant, mais la meilleure action *selon les valeurs courantes*. On apprend donc comme si, à partir de l'état suivant, on devenait immédiatement glouton.

Cela fait de Q-learning une méthode off-policy. On peut très bien collecter les données avec une politique exploratoire μ ε -gloutonne, et pourtant apprendre la valeur de la politique gloutonne π associée à Q . On peut donc dire que le schéma classique pour Q-learning consiste à prendre :

$$\pi(a | s) \in \arg \max_{a'} Q(s, a'), \quad \mu(\cdot | s) = \varepsilon\text{-gloutonne par rapport à } Q.$$

SARSA et Q-learning reprennent le même principe général que TD : après chaque transition observée, on corrige une estimation à partir d'une cible locale, puis on recommence. La différence essentielle ne porte donc pas sur la structure générale de l'algorithme, mais sur la quantité estimée et sur la forme de la cible utilisée dans la mise à jour. TD(0) met à jour une valeur d'état, tandis que SARSA et Q-learning mettent à jour des valeurs d'action. En outre, dans les méthodes de contrôle, il faut aussi choisir les actions au cours de l'apprentissage.

3.3.6 Garanties mathématiques : idée générale

Sous des hypothèses standards analogues à celles de SARSA (espaces finis, visites infinies des couples état-action, pas d'apprentissage appropriés) Q-learning converge vers q_* avec probabilité 1. La preuve complète est plus technique que dans un premier cours, mais l'idée de fond est claire : la mise à jour moyenne suit l'opérateur de Bellman optimal, qui est une contraction lorsque $\gamma < 1$.

3.3.7 SARSA versus Q-learning : l'exemple de la falaise

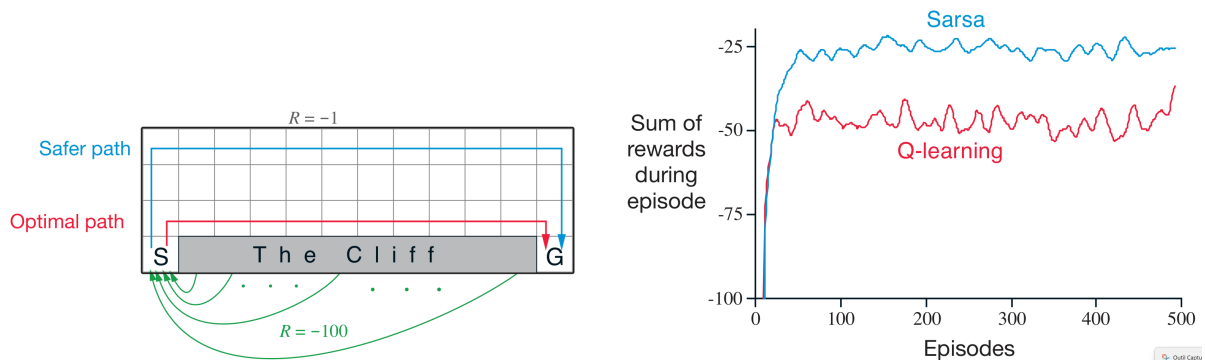
Un exemple classique pour comparer SARSA et Q-learning est le problème du *cliff walking* (marche au bord d'une falaise), présenté par Sutton et Barto [2, ex. 6.6]. Il s'agit d'un petit monde quadrillé rectangulaire. L'agent part de la case située en bas à gauche et doit atteindre la case but située en bas à droite. Les actions possibles sont les déplacements d'une case vers le haut, le bas, la gauche ou la droite. La tâche est épisodique et non actualisée ($\gamma = 1$). Chaque déplacement ordinaire reçoit une récompense

$$R = -1,$$

ce qui incite l'agent à atteindre le but en un nombre de pas aussi petit que possible. En revanche, les cases de la bande inférieure situées entre le départ et le but forment la *falaise*. Si l'agent entre dans cette région, il reçoit une forte pénalité

$$R = -100,$$

et il est immédiatement renvoyé au départ.



(a) Le problème de la falaise : chemin court au bord de la falaise et chemin plus sûr par le haut.

(b) Performance empirique de SARSA et Q-learning sur le problème de la falaise.

FIGURE 3.2 – L'exemple de la falaise, adapté de Sutton et Barto [2, ex. 6.6].

La première figur (à gauche) montre les deux comportements typiques appris par les deux algorithmes. Le chemin en rouge est le plus court : il longe directement la falaise. Si aucune erreur n'est commise, c'est bien le trajet optimal au sens du cumul des récompenses, car il minimise le nombre de pas et évite la pénalité -100 . C'est ce type de chemin que Q-learning tend à apprendre. Le chemin en bleu est plus long, donc un peu moins bon si l'on suppose que l'on suivra ensuite toujours les meilleures actions sans jamais dévier. En revanche, il est plus sûr, car il reste à distance de la falaise. C'est ce type de chemin que SARSA apprend souvent.

Cette différence s'explique par la nature même des deux méthodes. Q-learning utilise la cible

$$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'),$$

donc il évalue chaque action en supposant qu'à partir de l'état suivant on choisira toujours l'action meilleure selon l'estimation courante. Il apprend ainsi la valeur de la politique gloutonne, même si le

comportement réel pendant l'apprentissage reste exploratoire. SARSA, au contraire, utilise la cible

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}),$$

où A_{t+1} est l'action effectivement choisie par la politique suivie, souvent ε -gloutonne. Sa valeur tient donc compte du fait que, plus tard, une action d'exploration pourra être choisie.

Dans le problème de la falaise, cette différence devient très concrète. Lorsque l'agent longe la falaise, une seule action d'exploration malheureuse peut le faire tomber et provoquer la pénalité -100 . Or SARSA tient compte de ce risque, précisément parce qu'il apprend la valeur du comportement réel, exploration comprise. Il préfère donc souvent un chemin un peu plus long, mais plus robuste. Q-learning, lui, ne tient pas compte dans sa cible de ces futures actions exploratoires : il apprend la valeur du chemin optimal *si l'on se comporte ensuite de manière gloutonne*. Il tend donc à privilégier le bord de la falaise.

La seconde figure représente la somme des récompenses obtenues pendant chaque épisode, en fonction du numéro de l'épisode. Comme chaque pas ordinaire coûte -1 , des valeurs plus élevées (donc moins négatives) correspondent à de meilleures performances. On observe que la courbe de SARSA se stabilise au-dessus de celle de Q-learning. Cela signifie que, pendant l'apprentissage avec exploration persistante, SARSA obtient en moyenne de meilleurs épisodes. La raison est que sa politique apprise évite plus souvent les chutes dans la falaise. Q-learning, lui, vise un chemin théoriquement plus court, mais l'exploration ε -gloutonne continue à provoquer de temps en temps des chutes très coûteuses, ce qui dégrade la récompense cumulée observée.

Cet exemple met donc en évidence une idée importante. Q-learning apprend bien la valeur de la politique optimale, et il est donc naturel qu'il tende vers le chemin le plus court. Mais lorsque l'on regarde la performance *pendant* l'apprentissage, c'est-à-dire avec exploration encore active, cette stratégie peut être moins bonne en pratique qu'une stratégie plus prudente. SARSA, parce qu'il est *on-policy*, apprend précisément cette stratégie prudente. L'exemple de la falaise montre ainsi que la différence entre *on-policy* et *off-policy* n'est pas seulement une différence de formules : elle peut modifier qualitativement le comportement appris en présence de risque.

Enfin, si le paramètre d'exploration ε diminue progressivement vers 0, les deux méthodes finissent par se rapprocher d'un comportement glouton. La différence la plus visible entre elles apparaît donc surtout lorsque l'exploration reste présente pendant l'apprentissage.

3.3.8 Autres exemples classiques pour le TD de contrôle

Exemple 3.3 (Labyrinthe simple). Dans un petit labyrinthe avec récompense -1 à chaque pas et 0 à l'arrivée, SARSA et Q-learning apprennent tous deux à réduire la longueur moyenne des trajectoires. Q-learning tend à propager plus agressivement la meilleure continuation imaginable, tandis que SARSA reflète plus fidèlement le comportement réellement exécuté pendant l'apprentissage.

Exemple 3.4 (Grid world stochastique). Dans une grille stochastique, l'action choisie par l'agent n'est pas toujours exécutée exactement : par exemple, lorsqu'il veut avancer dans une direction, il peut avec une petite probabilité être dévié vers une case latérale. Cette incertitude peut rendre certaines zones ouvertes plus dangereuses que d'autres. À l'inverse, un couloir bordé de murs peut offrir une trajectoire plus sûre, car les déviations latérales y ont moins de conséquences. Dans un tel cadre, SARSA peut préférer un chemin un peu plus long mais protégé, tandis que Q-learning tend davantage à privilégier le chemin le plus court vers la sortie, malgré les risques. On retrouve ainsi une intuition proche de celle observée dans l'exemple de la falaise.

3.3.9 Vers l'approximation de fonction et le Deep RL

Jusqu'ici, on a supposé qu'il était possible de stocker explicitement une table $V(s)$ ou $Q(s, a)$ pour tous les états ou couples état-action. Cela devient irréaliste dès que l'espace d'états est très grand ou continu. On passe alors à la **value function approximation** : au lieu de mémoriser une valeur par état, on apprend une fonction paramétrée, par exemple linéaire ou neuronale,

$$q(s, a; \theta) \approx q_*(s, a).$$

Cette transition change l'échelle des problèmes accessibles. Elle permet d'envisager des états décrits par des vecteurs de grande dimension, des images, voire des séquences complexes.

Le pas conceptuel suivant mène au **Deep Q-Network (DQN)**, où un réseau de neurones profond reçoit directement une représentation riche de l'état, parfois presque brute, et produit des valeurs d'actions. On parle alors de **deep reinforcement learning**. Plusieurs raffinements classiques ont ensuite été introduits :

- **Double DQN**, qui réduit le biais d'optimisme induit par le maximum dans Q-learning ;
- **Dueling DQN**, qui sépare l'estimation d'une valeur d'état et d'un avantage relatif des actions ;
- **Deep RL end-to-end**, où la représentation utile à la décision est apprise directement à partir de l'entrée brute ;
- plus largement, des méthodes combinant replay buffer, réseaux cibles, approximation d'avantage, acteurs et critiques.

À ce stade, les idées du présent cours restent pourtant visibles. On retrouve toujours :

- une estimation de quantités de type valeur ;
- des mises à jour fondées sur des erreurs TD ;
- un compromis entre exploration et exploitation ;
- une tension entre on-policy et off-policy.

Autrement dit, le deep RL n'efface pas les méthodes élémentaires ; il les prolonge à grande échelle.

Conclusion

Le passage du model-based au model-free ne change pas l'objet fondamental du RL : on cherche toujours à estimer des valeurs et à construire de bonnes politiques dans un problème de décision séquentielle. Ce qui change, c'est la manière d'accéder à ces quantités. Au lieu de calculer des espérances exactes à partir d'un modèle connu, on les approxime à partir de trajectoires observées.

Les méthodes Monte Carlo exploitent des retours complets ; les méthodes TD exploitent des corrections locales par bootstrap ; les méthodes de contrôle comme SARSA et Q-learning ajoutent à cela la question de l'amélioration progressive de la politique. Derrière leur diversité apparente, toutes reposent sur la même idée : transformer l'expérience en corrections successives de valeurs, puis transformer ces valeurs en décisions de plus en plus pertinentes.

Ces méthodes constituent la base conceptuelle de développements plus avancés : approximation de fonction, actor-critic, politique par gradient, deep Q-learning, méthodes profondes continues et apprentissage à grande échelle. Pour comprendre ces techniques plus modernes, il est donc essentiel de bien maîtriser les mécanismes élémentaires étudiés ici.

Références

- [1] Marin Ferecatu, *Apprentissage par renforcement : cadre général et programmation dynamique*, notes de cours RL1, UE RCP211, LeCNAM, Paris, 2026.
- [2] Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning : An Introduction*, 2nd edition, The MIT Press, 2018.
<http://incompleteideas.net/book/the-book-2nd.html> (Licence : Creative Commons Attribution-NonCommercial-NoDerivs 2.0 Generic.)
- [3] Shiyu Zhao, *Mathematical Foundations of Reinforcement Learning*, Springer, 2026. Projet associé : <https://github.com/MathFoundationRL/Book-Mathematical-Foundation-of-Reinforcement-Learning>
- [4] David Silver, *Lectures on Reinforcement Learning*, 2015.
<https://www.davidsilver.uk/teaching/>
- [5] G. Tesauro, “Temporal Difference Learning and TD-Gammon,” *Communications of the ACM*, vol. 38, no. 3, pp. 58–68, 1995. doi :10.1145/203330.203343.
- [6] Christopher J. C. H. Watkins and Peter Dayan, “Q-learning”, *Machine Learning*, 8(3–4), 1992, p. 279–292.
- [7] Volodymyr Mnih et al., “Human-level control through deep reinforcement learning”, *Nature*, 518, 2015, p. 529–533.
- [8] Hado van Hasselt, Arthur Guez and David Silver, “Deep Reinforcement Learning with Double Q-learning”, *Proceedings of AAAI*, 2016.
- [9] Ziyu Wang et al., “Dueling Network Architectures for Deep Reinforcement Learning”, *Proceedings of ICML*, 2016.